Domain-Adapted Embedding Alignment Handbook

Implementation-level handbook for the current repository state and completed run artifacts.

Table of Contents

Project Goal and Scope

What it is
A local, reproducible pipeline to adapt embedding behavior for domain-sensitive retrieval tasks (medical, legal, cybersecurity) while preserving operational auditability.

Why it is used
Generic embeddings often underperform when domain vocabulary shifts from common language. This pipeline aligns embedding space around domain relevance signals and reports quality/latency tradeoffs end-to-end.

How it appears in code
- CLI command router: `src/domain_adapted_embedding_alignment/cli.py`
- Full orchestration: `src/domain_adapted_embedding_alignment/pipelines/run_end_to_end.py`

Practical implementation view
Each stage emits persisted artifacts consumed by downstream stages and by completion checks. This design makes runs inspectable and restartable at stage boundaries.

End-to-End Workflow

What it is
Stage sequence used by `dea-cli run-all`:
1. `prepare-data`
2. `train`
3. `evaluate`
4. `rag-benchmark`
5. `graphrag-benchmark`
6. `inference`
7. `build-final-report`
8. `finalize-reports`

Why it is used
Separate stages reduce blast radius, support partial reruns, and keep evidence traceable in `artifacts/`.

How it appears in code
- Stage orchestration: `src/domain_adapted_embedding_alignment/pipelines/run_end_to_end.py`
- Stage implementations: `src/domain_adapted_embedding_alignment/pipelines/*.py`

Practical implementation view
The same stage implementations can be called independently from CLI for targeted debugging and

benchmarking.

Configuration and Runtime Profiles

What it is
Typed runtime config via Pydantic settings with `DEA_` environment overrides.

Why it is used
Keeps behavior reproducible and hardware-aware without editing source files.

How it appears in code
- Config model: `src/domain_adapted_embedding_alignment/settings.py`
- Profiles: `configs/local_6gb.yaml`, `configs/smoke_cpu.yaml`

Practical implementation view
Key run values in the completed run:
- `final_pair_target=200000`
- `train_max_steps=700`
- `eval_query_limit=120`
- `rag_query_limit=120`
- `graphrag_query_limit=150`

Data Preparation

What it is
Loads real corpora and standardizes them into canonical parquet artifacts.

Why it is used
Reliable training/evaluation requires unified schema and deterministic preprocessing across heterogeneous sources.

How it appears in code
- Pipeline entry: `src/domain_adapted_embedding_alignment/pipelines/prepare_data.py`
- Loaders: `src/domain_adapted_embedding_alignment/data/loaders.py`
- Typed records: `src/domain_adapted_embedding_alignment/schemas.py`

Practical implementation view (run outputs)
From `artifacts/reports/data_preparation_report.json`:
- `documents_total`: 168448
- `queries_total`: 98907
- `pairs_total`: 200000
- `documents_by_domain`: general 153637, medical 4392, legal 7999, cybersecurity 2420

Pair Mining and Split Strategy

What it is
Contrastive pair construction with positive, random negative, and hard negative text per query.

Why it is used
Hard negatives are required to learn meaningful boundaries in embedding space; deterministic split assignment prevents accidental leakage.

How it appears in code
- Builder: `src/domain_adapted_embedding_alignment/data/pair_builder.py`
- Split function: `_split_from_query_id`
- Hard negative function: `_pick_hard_negative`
- Domain quotas: `_domain_quota`

Practical implementation view
Current implementation uses lexical-overlap sampled hard negatives with caching, avoiding full-corpus per-pair BM25 rescoring at 200k-pair scale.

Training Backends: PEFT and Unsloth

What it is
Backend routing chooses either:
- PEFT path (`ContrastiveTrainer`)
- Unsloth path (`UnslothContrastiveTrainer` with SentenceTransformers trainer)

Why it is used
Allows stable fallback behavior while exploiting acceleration when environment supports Unsloth.

How it appears in code
- Resolver: `src/domain_adapted_embedding_alignment/training/backend_selection.py`
- Training entry: `src/domain_adapted_embedding_alignment/pipelines/train_model.py`
- Unsloth trainer: `src/domain_adapted_embedding_alignment/training/unsloth_trainer.py`
- PEFT trainer: `src/domain_adapted_embedding_alignment/training/trainer.py`

Practical implementation view (run outputs)
From `artifacts/reports/training_report.json`:
- `runtime.backend_requested`: `unsloth`
- `runtime.backend_used`: `unsloth`
- `runtime.fallback_reason`: `null`
- `total_steps`: 700
- `best_val_loss`: 0.06946182250976562
- `train_runtime_seconds`: 5120.9695

Retrieval Stack: BM25, Dense, Hybrid

What it is
Five retrieval systems are evaluated:
- BM25 sparse retrieval
- Dense baseline (`qwen3-embedding:4b`)
- Dense tuned (`Qwen3-Embedding-0.6B` + adapter)
- Hybrid baseline
- Hybrid tuned

Why it is used
Retrieval quality and latency profiles vary sharply between sparse and dense methods; hybrid fusion can stabilize performance.

How it appears in code
- Backend construction: `src/domain_adapted_embedding_alignment/retrieval/backend_factory.py`
- Embedding backends: `src/domain_adapted_embedding_alignment/retrieval/embeddings.py`
- BM25: `src/domain_adapted_embedding_alignment/retrieval/bm25.py`
- Dense retrieval: `src/domain_adapted_embedding_alignment/retrieval/dense.py`
- Fusion: `src/domain_adapted_embedding_alignment/retrieval/hybrid.py`

Practical implementation view (run outputs)
From `artifacts/evaluation/retrieval_evaluation.json`:
- BM25 recall@10 0.9667, mean latency 0.86ms
- Baseline dense recall@10 1.0000, mean latency 2902.25ms
- Tuned dense recall@10 0.9750, mean latency 47.33ms
- Hybrid tuned recall@10 0.9833, mean latency 56.04ms

Interpretation: this run shows a strong quality-latency tradeoff rather than a single global winner.

Evaluation, Judging, and Diagnostics

What it is
Evaluation layer combining retrieval metrics, latency summaries, RAG proxies, optional LLM judge scoring, and embedding-space diagnostics.

Why it is used

Production retrieval decisions require both quality and operational evidence.

How it appears in code
- Evaluation pipeline: `src/domain_adapted_embedding_alignment/pipelines/evaluate_models.py`
- Core evaluator: `src/domain_adapted_embedding_alignment/evaluation/evaluator.py`
- Judge fallback logic: `src/domain_adapted_embedding_alignment/evaluation/llm_judge.py`
- Metric helpers: `src/domain_adapted_embedding_alignment/retrieval/metrics.py`
- Embedding projections: `src/domain_adapted_embedding_alignment/evaluation/visualization.py`

Practical implementation view (run outputs)
From `artifacts/evaluation/retrieval_evaluation.json`:
- Similarity ranking accuracy improved from 0.8933 to 0.9700 (baseline -> tuned)
- Judge metrics are present for baseline and tuned dense systems

From `artifacts/logs/02_evaluate.log`:
- Multiple judge parse failures were logged for `qwen3.5:4b`
- Fallback scores remained neutral by design, keeping evaluation execution stable

RAG Benchmarking (Chroma + Pinecone)

What it is
RAG benchmark layer over local Chroma collections plus optional Pinecone path.

Why it is used
RAG retrieval behavior can differ from standalone retrieval benchmarking due to indexing/runtime interactions.

How it appears in code
- Pipeline: `src/domain_adapted_embedding_alignment/pipelines/run_rag_benchmarks.py`
- Chroma helpers: `src/domain_adapted_embedding_alignment/rag/chroma_demo.py`
- Pinecone helpers: `src/domain_adapted_embedding_alignment/rag/pinecone_demo.py`

Practical implementation view (run outputs)
From `artifacts/evaluation/rag_benchmark.json`:
- Chroma baseline hit@5: 1.0
- Chroma tuned hit@5: 0.9416666666666667
- Chroma mean latency: 159.95ms baseline vs 45.06ms tuned
- Pinecone: `{ "executed": false, "reason": "disabled" }`

GraphRAG Benchmarking

What it is
Graph-enhanced retrieval using a document-term graph with local neighborhood expansion and global community retrieval.

Why it is used
Graph traversal provides structural context beyond plain nearest-neighbor lookup.

How it appears in code
- Graph construction: `src/domain_adapted_embedding_alignment/graphrag/graph_builder.py`
- Local/global retrieval: `src/domain_adapted_embedding_alignment/graphrag/retriever.py`
- Benchmark pipeline: `src/domain_adapted_embedding_alignment/pipelines/run_graphrag_benchmarks.py`

Practical implementation view (run outputs)
From `artifacts/logs/04_graphrag.log`:
- Graph built with 9220 nodes, 187729 edges, 6 communities

From `artifacts/evaluation/graphrag_benchmark.json`:
- Baseline local/global hit@5: 0.9467 / 0.46
- Tuned local/global hit@5: 0.88 / 0.46
- Query count: 150

Inference Path

What it is
Query-time retrieval endpoint returning ranked results with metadata and previews.

Why it is used
Represents end-user retrieval behavior after training and indexing stages.

How it appears in code
- Inference stage: `src/domain_adapted_embedding_alignment/pipelines/run_inference.py`
- Output formatting: `src/domain_adapted_embedding_alignment/rag/inference.py`

Practical implementation view (run outputs)
From `artifacts/logs/05_inference.log`:
- Example queries executed: 3
- Mean latency: 65.10ms
- Backend used: `HuggingFaceEmbeddingBackend`
- Outputs include `rank`, `doc_id`, `score`, `domain`, `source`, `preview`

Final Reporting and Completion Gates

What it is
Final aggregation and validation layer that determines run completeness.

Why it is used
Prevents ambiguous "done" status by checking artifacts and runtime metadata explicitly.

How it appears in code
- Final report assembly: `src/domain_adapted_embedding_alignment/pipelines/build_final_report.py`
- Completion/tool impact reports: `src/domain_adapted_embedding_alignment/pipelines/finalize_reports.py`

Practical implementation view (run outputs)
From `artifacts/reports/completion_checklist.json`:
- `all_required_artifacts_ok`: true
- `all_runtime_checks_ok`: true
- `project_completion_ok`: true

From `artifacts/reports/tool_impact_report.json`:
- `unsloth.used`: true
- `peft.used`: false
- `trl.used_in_runtime`: false

Testing and Reliability

What it is
Unit-level checks for retrieval metrics, backend selection, pair splits, report finalization, and notebook scaffolding.

Why it is used
Protects correctness in core evaluation math and completion criteria.

How it appears in code
- Test suite: `tests/`

Practical implementation view (run outputs)
From `artifacts/logs/09_pytest.log`:
- `12` tests passed.

Operational Caveats

- Latency values are hardware- and profile-dependent.
- LLM judge failures do not crash evaluation; fallback scores are neutral.

- Pinecone path is optional and controlled by runtime settings.
- Baseline dense and tuned dense may trade quality for latency differently by domain and query distribution.

References

Project-local references
- `README.md`
- `docs/tooling_decisions.md`
- `artifacts/reports/final_report.json`
- `artifacts/reports/completion_checklist.json`
- `artifacts/logs/`

External references
- Unsloth embedding fine-tuning: https://unsloth.ai/docs/basics/embedding-finetuning
- PEFT task types (`FEATURE_EXTRACTION`): https://huggingface.co/docs/peft/main/package_reference/peft_types
- TRL overview: https://huggingface.co/docs/trl
- Sentence Transformers losses: https://www.sbert.net/docs/package_reference/sentence_transformer/losses.html
- Ollama embed API: https://docs.ollama.com/api/embed
- Chroma docs: https://docs.trychroma.com/
- Pinecone docs: https://docs.pinecone.io/
- Microsoft GraphRAG docs: https://github.com/microsoft/graphrag/blob/main/docs/query/overview.md